

An empirical study of system design instability metric and design evolution in an agile software process

Mohammad Alshayeb ^{a,*}, Wei Li ^b

^a *Information and Computer Science Department, King Fahd University of Petroleum and Minerals, P.O. Box 1172, Dhahran 31261, Saudi Arabia*

^b *Computer Science Department, University of Alabama in Huntsville, Huntsville, AL 35899, USA*

Received 2 June 2003; received in revised form 28 January 2004; accepted 1 February 2004

Available online 19 March 2004

Abstract

Software project tracking and project plan adjustment are two important software engineering activities. The class growth shows the design evolution of the software. The System Design Instability (SDI) metric indicates the progress of an object-oriented (OO) project once the project is set in motion. The SDI metric provides information on project evolution to project managers for possible adjustment to the project plan. The objectives of this paper are to test if the System Design Instability metric can be used to estimate and re-plan software projects in an XP-like agile process and study system design evolution in the Agile software process. We present an empirical study of the class growth and the SDI metric in two OO systems, developed using an agile process similar to Extreme Programming (XP). We analyzed the system evolutionary data collected on a daily basis from the two systems. We concluded that: the systems' class growth follows observable trends, the SDI metric can indicate project progress with certain trends, and the SDI metric is correlated with XP activities. In both of the analyzed systems, we observed two consistent jumps in the SDI metric values in early and late development phases. Part of the results agrees with a previous empirical study in another environment.

© 2004 Elsevier Inc. All rights reserved.

Keywords: Empirical study; System design instability (SDI) metric; Design evolution in agile software process; Extreme programming

1. Introduction

Monitoring and controlling software projects are important tasks in software engineering. The Capability Maturity Model (CMM) identifies Project Tracking as a Key Process Area (KPA) (Paulk et al., 1993). One key factor in effective software project management is fast and accurate feedback during the actual design process once the project starts. Because most software projects deviate from the original plan, it is helpful to quantitatively measure the actual project progress so that the project plan can be adjusted accordingly. For example, if an integration test is scheduled to occur in two weeks

in the original Project Plan, but the project feedback indicates that the project is still in an unstable stage and will not be ready for integration, the project manager can then adjust the integration test plan to avoid potential problems.

Many studies evaluated quantitative software product metrics in the object-oriented paradigm (Li and Henry, 1993a; Chidamber and Kemerer, 1994; Basili et al., 1996a; Briand et al., 1999). Metrics were used in conjunction with prediction models to predict different software aspects such as effort (Basili et al., 1996b; Nesi and Querci, 1998; Moser et al., 1999; Hastings and Sajeev, 2001; Briand and Wüst, 2001; Alshayeb and Li, 2003). Researchers have also tried to predict maintenance (Rombach, 1987; Rombach, 1990; Li and Henry, 1993b; Fioravanto and Nesi, 2001). Other researchers questioned the predictive capabilities of metrics and prompted questions about their use (Fenton and Neil,

* Corresponding author. Tel.: +966-3860-4874; fax: +966-3860-1562.

E-mail addresses: mohammad@alshayeb.com (M. Alshayeb), wli@cs.uah.edu (W. Li).

1999; Fenton and Ohlsson, 2000). However, these OO metrics measure software products after they are designed or produced, rather than capture the change pattern of products during development.

The class growth and the SDI metric can help monitor an OO project's progress once the project starts. The class growth shows the design evolution of the software by showing the number of classes in the system at a given point. The System Design Instability (SDI) metric measures the system-level design changes in an OO system; these changes mainly reflect designers' shifting views in domain abstraction.

2. Instability metric

As part of an OO design process, developers decide the number of classes, their names, and their organization in the inheritance hierarchy in the systems. The changes in class names and inheritance hierarchy, addition of new classes, and deletion of existing classes all indicate the stability/instability of a design and the abstraction of an application domain. In other words, if the domain abstraction and design are stable, the number of classes, class names, and the inheritance hierarchy in an application are likely to stay stable as well. Therefore, by measuring the changes in these factors during development, we obtain a measure of the system design stability/instability at the abstraction/system level.

Li et al. proposed the System Design Instability (SDI) metric to provide project progress feedback. The SDI metric measures the system-level design changes from design N to design $N + 1$. It combines four types of changes in an OO system—percentage of classes whose names have changed, percentage of added classes, percentage of deleted classes, and percentage of classes whose inheritance hierarchy have changed (Li et al., 2000). The amount of change is expressed in percentages so the magnitudes of the values will be comparable.

2.1. System design instability

Definition: The SDI metric measures the percentage of classes in Design N that are affected by changes in Design $N + 1$. The SDI metric is defined as follows:

SDI = percentage of classes whose names have changed + percentage of newly added classes + percentage of deleted classes + percentage of classes whose inheritance hierarchy have changed at the time when Design $N + 1$ is registered in the configuration management system, the SDI metric is $[(a + b + c + d)/m] * 100$; where:

- m : number of classes in Design N , where m is a positive integer.

- a : Number of classes whose names have changed from Design N to Design $N + 1$, where a is a positive integer.
- b : Number of newly added classes in Design $N + 1$, where b is a positive integer.
- c : Number of deleted classes from Design N , where c is a positive integer.
- d : Number of classes whose inheritance hierarchy have changed from Design N to Design $N + 1$, where d is a positive integer.

The SDI metric value can be zero or a positive number. The unit for the SDI metric is the “percentage of the classes in Design N ” (Li et al., 2000).

3. Extreme programming

We developed two Java systems using an agile software process (Ambler, 2002) that is similar to Extreme Programming (XP) (Beck, 1999). Beck introduced XP as a new iterative software process that differs from other processes. The XP process is especially convenient and effective for projects that have vague or changing requirements. The XP development process has various unique features such as stories, pair programming, test-driven design, frequent unit testing, and continuous integration. XP emphasizes short iteration cycles, which are triggered by stories. A story is a set of requirements supplied by users/customers. XP is a test-driven iterative process. Test case design precedes the architectural and system design in each iteration cycle (Beck, 1999).

There are three main activities in each XP iteration cycle: *new design*, *error fix*, and *refactoring*. Adding new classes or methods are considered new design. Error-fix corrects deficiencies uncovered during the development. Refactoring changes system architecture because programmers believe that the new architecture is better based on their experience.

4. Research objectives

In this study, we investigate if the class the growth in both systems shows any observable trend. From past experience and a previous study (Li et al., 2000), we believe that the class growth can show observable trend. We also examine whether the SDI metric can indicate design progress and how system instabilities relate to the different efforts in the XP cycles. Also from past experience and a previous study (Li et al., 2000), we believe that the SDI metric can indicate design progress with observable patterns such as gradual decline in the SDI value as a project evolves. We also speculate that the SDI metric should be positively correlated with all activities in the agile process because the less stable a

system is (high SDI value), the more XP activities are expected. To focus our efforts, we set up five hypotheses as follows:

- Hypothesis 1: The class growth in systems 1 and 2 follows observable trends.
- Hypothesis 2: The SDI metric can indicate design progress with observable patterns.
- Hypothesis 3: The instability of system design is positively linked with new design effort.
- Hypothesis 4: The instability of system design is positively linked with the refactoring effort.
- Hypothesis 5: The instability of system design is positively linked with the error-fix effort.

We expect to confirm the findings of previous studies that the class growth follows observable trends and the SDI metric can indicate design progress with observable patterns. We anticipate positive correlation between new design and SDI because adding new design components requires effort, so instability and effort should move in the same direction. We also expect positive correlation between refactoring and SDI because changing the system architecture will cause the system to be unstable, and positive correlation between error fix and SDI because the more error fix done to the classes the more changes made to the system, thus the more unstable the system is.

We collected system evolution data from two client-server systems; data collection from the first system happened over a period of 18 months, while data collection from the second system happened over a period of 6 months. The two systems were developed primarily using Java, Java Servlets and XML. The two systems were delivered as commercial products to customers. Both systems were developed using an agile process similar to XP with several short iteration cycles and customer-supplied stories; each story marked the beginning of a new cycle. The programmers/designers planned and designed for a story at beginning of each cycle. These two projects were the first experience for the developers in applying XP process. The developers were comfortable in applying the as well as apply XP activities such as refactoring, error fix and new design.

Tables 1 and 2 show the number of classes (added, deleted, renamed) in each story for System 1 and 2

Table 2

System 2: number of classes in each story

Story number	Added classes	Deleted classes	Changes in class names	Total number of classes
Story 1	2	0	0	2
Story 2	3	1	1	4
Story 3	5	2	1	7
Story 4	14	4	1	17
Story 5	2	0	2	19
Story 6	34	1	3	52
Story 7	10	12	0	50

respectively. No classes in the two systems changed their inheritance levels.

We collected system evolution data based on research logs. Research logs recorded daily activities when the two systems were developed. A log file was created for each working day during system development. The recorded information in the log files include:

- The tasks planned for the current iteration cycle.
- A description of the progress/non-progress towards completing the tasks.
- The time spent on each task measured by hours.
- A description of the problems encountered during implementing a task, and the attempted solutions.
- A description of the changes made on the system, the reasons for the changes, and the affected classes.

5. Research results

To test the first and the second hypotheses, we referenced the result from another study (Li et al., 2000) where the evolution of an OO system design revealed an increasing trend for the number of classes and somewhat declining SDI metrics.

5.1. Class growth

Figs. 1 and 2 show the class growth across all cycles in the two systems respectively. We observed a similar trend—a relatively consistent growth before reaching the peak with a dip at the end—in both systems. This observation agrees with the finding in a previous study

Table 1

System 1: number of classes in each story

Story number	Added classes	Deleted classes	Changes in class names	Total number of classes
Story 1	1	0	0	1
Story 2	2	0	0	3
Story 3	5	2	0	6
Story 4	2	0	1	8
Story 5	11	0	1	19
Story 6	14	3	2	30

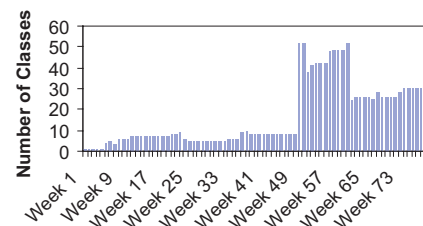


Fig. 1. System 1: number of classes (weekly).

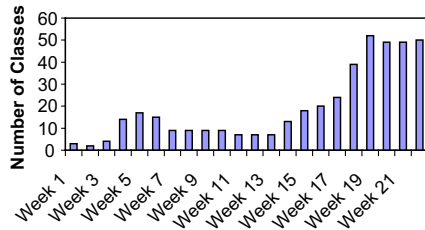


Fig. 2. System 2: number of classes (weekly).

(Li et al., 2000). We also noticed that from the low class counts, after the projects moved beyond their initial phases (first several weeks), to the final class counts, there was more than 600% growth in both systems (600% and 625% in System 1 and 2 respectively). The significant class growth occurred either before or after the last quarter mark of the projects. Although the 600% class growth rate may be limited to the contexts of the two observed projects, we think that the significant class growth around the beginning of the project's last quarter may be a common phenomenon. This is because a large number of *implementer classes* can be added late in the project; we consider classes that mainly help the implementation of an OO project (*implementer classes*) because they do not have implications in the domain abstraction nor the system architecture. We thus accept hypothesis 1: the class growth in systems 1 and 2 follows observable trends.

Figs. 3 and 4 show the class growth based on each iteration cycle (marked by a story) in the two systems respectively. The relatively smooth growth trends are more similar in the short-iteration cycles than across the cycles.

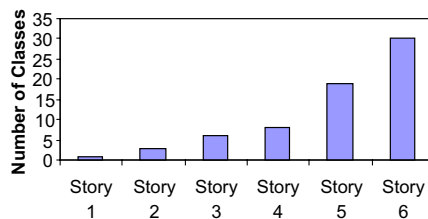


Fig. 3. System 1: number of classes (stories).

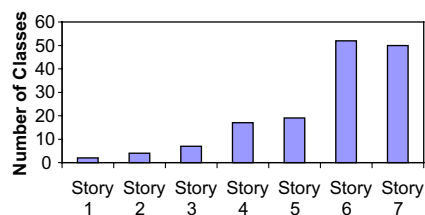


Fig. 4. System 2: number of classes (stories).

5.2. System design instability

Figs. 5 and 6 show the evolution of the SDI metric for the two systems across all iteration cycles. Figs. 7 and 8 show the evolution of the SDI metric based on each iteration cycle, which is marked by a story.

We were expecting to see gradually decreasing instability in design, thus decreasing SDI values. Although there is a general decreasing trend in Figs. 5 and 6, the trend is not very clear. However, we observed two spikes in the SDI metric in both systems: one at the early stage

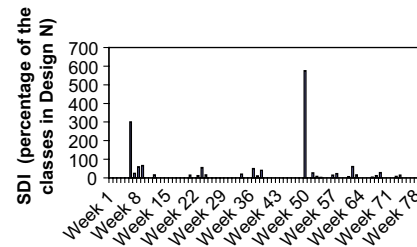


Fig. 5. System 1: system design instability (weekly).

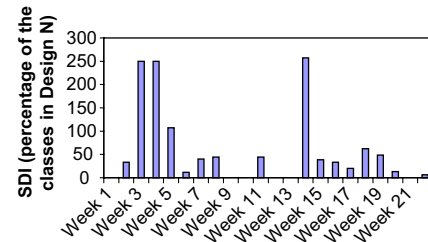


Fig. 6. System 2: system design instability (weekly).

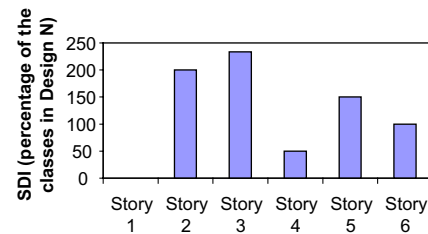


Fig. 7. System 1: system design instability (stories).

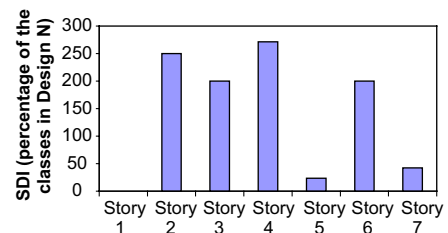


Fig. 8. System 2: system design instability (stories).

of the development, the other in the late phase. We further noticed that the first spike occurred before the first quarter and the second soon around the fourth-quarter mark in both systems. We then examined the data from a previous study and surprisingly found a similar trend, although the second SDI spike in the previous study is not as clear and high as in System 1 and 2. We summarize the timings of the spikes in all three systems with System 3 being the system in the previous study (Li et al., 2000).

The first spike can be explained by the fact that the beginning of system development includes considering various design alternatives, thus making the system unstable at that stage; once a specific system design is adopted, the system returns to stability.

The second spike is implementation related. It can be explained by the fact that the first half of project development includes building the basic architecture of the most valuable stories in the system; implementation details are usually avoided in the first half of the project. Towards the end of the project, implementation specifics and details are added to the system and finalized, thus explaining the instability near the end of development.

The numerators in Table 3 data indicate when the SDI spikes occurred. The denominators are the total project durations, measured in *weeks* in the first two systems and *data points* in the third system; the distance between two data points can be one or several working days. The SDI spikes occurred in the two systems and

Table 3
When the SDI spikes occurred

	First SDI spike	Second SDI spike	Project duration
System 1	6/63 (10%)	50/63 (80%)	63 weeks
System 2	4/20 (20%)	14/20 (70%)	20 weeks
System 3	4/17 (24%)	13/17 (76%)	17 data points

their timing is a surprising empirical finding; it can help managers of agile processes better gauge a project progress by using the occurrence of SDI spikes as milestones. The managers can use the identified first SDI spike as an indication that the OO project is approaching the first quarter mark in the project. The second SDI jump marks the beginning of the fourth quarter of the project. We thus accept the second hypothesis: the SDI metric can indicate design progress with observable patterns.

5.3. The correlation between SDI and XP activities

Tables 4 and 5 show the correlations between the SDI metric and effort data for stories in System 1 and 2 respectively. The highly significant positive correlation between the SDI metric and the *new-design* effort almost in all stories confirms our speculation that the more new designs are added to the system, the more unstable the system architecture is, thus accepting Hypothesis 3.

Table 4
System 1: correlation between SDI and XP activities effort

	Sample size Number of Weeks	SDI and new design		SDI and refactoring		SDI and error fix	
		P-value	Pearson correlation	P-value	Pearson correlation	P-value	Pearson correlation
Story 1	5	Not enough data points					
Story 2	3	0.049	0.997	0.041	−0.998	0.038	−0.827
Story 3	17	0.000	0.922	0.000	−0.788	0.091	−0.423
Story 4	13	0.000	0.964	0.005	−0.721	0.007	−0.709
Story 5	11	0.000	0.998	0.002	−0.828	0.041	−0.622
Story 6	14	0.000	0.866	0.001	−0.788	0.003	−0.73
All stories	63	0.000	0.865	0.072	−0.228	0.020	0.292

Table 5
System 2: correlation between SDI and XP activities effort

	Sample size Number of weeks	SDI and new design		SDI and refactoring		SDI and error fix	
		P-value	Pearson correlation	P-value	Pearson correlation	P-value	Pearson correlation
Story 1	2	Not enough data points					
Story 2	1	Not enough data points					
Story 3	8	0.003	0.894	0.044	−0.721	0.251	−0.460
Story 4	1	Not enough data points					
Story 5	2	Not enough data points					
Story 6	3	0.099	0.988	0.106	−0.986	0.006	−1.000
Story 7	3	0.073	0.993	0.259	−0.918	0.040	−0.998
All stories	20	0.019	0.520	0.186	−0.308	0.879	−0.036

The highly significant negative correlations between the SDI metric and *refactoring* as well as *error-fix* efforts almost in all stories surprise us. We were expecting positive correlations between them as we stated in Hypotheses 4 and 5, which are soundly rejected by the empirical data. This observation can be explained; increased refactoring and error-fix efforts indicate a relatively mature and stable design at the domain-abstraction level, which is the design level that the SDI metric measures and also error-fixes occur inside classes, and are thereby not accounted for by the SDI.

However, we find no correlation between the SDI metric with any effort if we break the story boundaries. This indicates to us that each story is a relatively independent and self-contained development cycle in terms of the three XP activities.

6. Conclusion

Tracking software project progress and adjusting the project plan accordingly are important aspects of software engineering. From our empirical study, we found that the class growth in the systems follows some observable trends; we observed a significant class increase toward the fourth quarter in both projects. We also found that the SDI metric may possibly indicate project progress in an agile software process; and thus provides information on project evolution to project managers for possible adjustment to the project plan. We also compared our observations as a validation study with another study that followed the traditional software process and we found that the products developed using traditional software process and XP follow the same pattern. Thus we conclude that the SDI metrics can be used to estimate and re-plan software projects in traditional process as well as in an XP-like agile process. This particularly clear in the first and fourth quarters, where increased system instability was indicated by spikes in the SDI metric values consistently in the two analyzed systems. We also discovered that the SDI metric is positively correlated with the new-design effort and negatively correlated with the refactoring and error-fix efforts in almost all short cycles in an agile process, if sufficient data were present. We believe that our conclusions are limited to relatively short-cycled iterative process and projects that a team does not have prior experiences in developing similar projects in the same process environment, so if the development team has extensive experience doing similar projects, or if the process is not a short-cycled agile process, our conclusions may not apply.

Although our study used empirical data from two OO systems, we think that the observations from the study provide reasonable basis for further validation studies.

Acknowledgements

The authors would like to thank the anonymous reviewers for their constructive contributions. This work is supported in part by a grant from National Science Foundation (NSF CCR-0097058).

References

- Alshayeb, M., Li, W., 2003. An empirical validation of object-oriented metrics in two iterative processes. *IEEE Transactions on Software Engineering* 29 (11), 1043–1049.
- Ambler, S.W., 2002. *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*. Wiley.
- Basili, V., Briand, L., Melo, W.L., 1996a. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering* 22 (10), 267–271.
- Basili, V., Briand, L., Condon, S., Kim, Y., Melo, W., Valett, J., 1996. Understanding and Predicting the Process of Software Maintenance Releases. In: *Proceedings of the 8th International Conference on Software Engineering (ICSE-18)*, Berlin, Germany, March, pp. 25–29.
- Beck, K., 1999. Embracing changes with Extreme programming. *IEEE Computer* (October), 70–77.
- Briand, L., Morasca, S., Basili, V., 1999. Defining and validating measures for object-based high-level design. *IEEE Transactions on Software Engineering* 25 (6), 722–743.
- Briand, L., Wüst, J., 2001. Modeling development effort in object-oriented systems using design properties. *IEEE Transactions on Software Engineering* 27 (11), 963–986.
- Chidamber, S.R., Kemerer, C.F., 1994. A metrics suite for object-oriented design. *IEEE Transactions on Software Engineering* 20 (6), 476–493.
- Fenton, N., Neil, M., 1999. A critique of software defect prediction models. *IEEE Transactions on Software Engineering* 25 (5), 675–689.
- Fenton, N., Ohlsson, N., 2000. Quantitative analysis of faults and failures in a complex software system. *IEEE Transactions on Software Engineering* 26 (8), 797–814.
- Fioravanto, F., Nesi, P., 2001. Estimation and prediction metrics for adaptive maintenance effort of object-oriented systems. *IEEE Transactions on Software Engineering* 27 (12), 1062–1084.
- Hastings, T., Sajeev, A., 2001. A vector-based approach to software size measurement and effort estimation. *IEEE Transactions on Software Engineering* 27 (4), 337–350.
- Li, W., Etzkorn, L., Davis, C., Talburt, J., 2000. An empirical study of object-oriented system evolution. *Information and Software Technology (Elsevier journal)* 42 (6), 373–381.
- Li, W., Henry, S., 1993a. Object oriented metrics that predict maintainability. *Journal of Systems and Software* 23 (2), 111–122.
- Li, W., Henry, S., 1993b. Maintenance metrics for object-oriented paradigm. In: *Proceedings of the First International Software Metrics Symposium*, May, pp. 52–60.
- Moser, S., Henderson-Sellers, B., Mišić, V., 1999. Cost estimation based on business models. *Journal of Systems and Software* 49 (1), 33–42.
- Nesi, P., Querci, T., 1998. Effort estimation and prediction of object-oriented systems. *Journal of Systems and Software* 42 (1), 89–102.
- Paulk, M., Curtis, B., Chrissis, M., Weber, C., 1993. *Capability Maturity Model for software*. Software Engineering Institute Carnegie Mellon University, Pittsburgh, PA.
- Rombach, H., 1987. A controlled experiment on the impact of software structure on maintainability. *IEEE Transactions on Software Engineering* 13 (3), 334–354.
- Rombach, H., 1990. Design measurement, some lessons learned. *IEEE Software* 7 (2), 17–25.